

Prompt:

What are good hyperparameter test values for a yolo object detection model?

Chat GPT said:

Great — tuning hyperparameters for a **YOLO (You Only Look Once)** object detection model can dramatically affect performance. The best values depend on your dataset (size, object scale, lighting, etc.), but there are some well-tested **starting ranges** used by practitioners and Ultralytics' own tuning framework.

Below is a guide covering the main categories:

⚙️ 1. Training Configuration

Parameter	Typical Range / Default	What It Does
epochs	100–300	Number of full passes through the dataset. Small datasets often need more epochs.
batch	16–64	Number of images per batch. Larger batches = more stable gradients but require more GPU memory.
imgsz	640 (range: 416–1280)	Input image size (square). Larger → better accuracy, slower training.
workers	4–8	Number of dataloader threads.

💡 Start with `imgsz=640`, `batch=16`, `epochs=200`.

🎯 2. Optimizer and Learning Rate

Parameter	Recommended Range	Notes
optimizer	"SGD" or "AdamW"	AdamW often converges faster; SGD more stable.
lr0 (<i>initial LR</i>)	0.001 – 0.01	Smaller for fine-tuning; larger for training from scratch.
lrf (<i>final LR fraction</i>)	0.01 – 0.2	Controls learning rate decay schedule.
momentum	0.8 – 0.98	Controls optimizer smoothing.
weight_decay	0.0005 – 0.001	Regularization term.

💡 Typical combo: AdamW, lr0=0.002, lrf=0.01, weight_decay=0.0005.

📦 3. Data Augmentation

Augmentation is **hugely influential** for YOLO models — especially on smaller datasets.

Parameter	Typical Range	Description
hsv_h / hsv_s / hsv_v	0.015 / 0.7 / 0.4	Color jitter: hue, saturation, brightness.
degrees	0 – 10	Rotation.
translate	0 – 0.1	Random image translation.
scale	0.5 – 1.5	Random scaling of objects.
shear	0 – 2	Shearing transformation.
flipud / fliplr	0.0 / 0.5	Vertical/horizontal flips.
mosaic	1.0	Enables mosaic augmentation (combining 4 images).
mixup	0.1 – 0.3	Overlaying images for regularization.

💡 Start with the Ultralytics default augmentations (already well-tuned). Reduce mosaic/mixup for smaller datasets.

📊 4. Loss Function Hyperparameters

Parameter	Range	Description
box	0.02 – 0.05	Weight for bounding box regression loss.
cls	0.2 – 0.7	Weight for classification loss.
dfi	1.0	Distribution focal loss weight (YOLOv8+).
obj	0.3 – 1.0	Objectness loss weight.

💡 If model overfits classification (too many false positives), lower *cls* slightly.

🧠 5. Advanced: Scheduler & Warmup

Parameter	Range / Default	Description
warmup_epochs	2–5	Gradually increases LR to prevent early instability.

cos_lr	True	Cosine learning rate schedule.
ema	True	Exponential moving average of weights → smoother convergence.

6. Example YOLOv8/YOLOv5 Training Command

- `yolo detect train data=data.yaml model=yolov8m.pt epochs=200 imgsz=640 batch=16 \`
- `lr0=0.002 lrf=0.01 momentum=0.937 weight_decay=0.0005 \`
- `hsv_h=0.015 hsv_s=0.7 hsv_v=0.4 degrees=0 translate=0.1 scale=0.9 shear=0.0 \`
- `flipud=0.0 fliplr=0.5 mosaic=1.0 mixup=0.2`

7. Automated Hyperparameter Search

Ultralytics YOLO supports built-in tuning:

- `yolo tune model=yolov8m.pt data=data.yaml epochs=50 imgsz=640`

It uses **Bayesian optimization** to explore hyperparameters like `lr0`, `momentum`, `weight_decay`, `scale`, etc., and logs best configurations automatically.

8. Good Default Baselines by Model Size

Model	Batch	LR	Epochs	Image Size	Notes
YOLOv8n (nano)	64	0.01	100	640	Fast experiments
YOLOv8s (small)	32	0.005	150	640	Good balance
YOLOv8m (medium)	16	0.002	200	640–800	Strong baseline
YOLOv8l/x (large)	8	0.001	200	800–1280	Heavy compute needs

Would you like me to give you a **Colab script** that automatically runs a small **hyperparameter sweep** (say, on `lr0`, `momentum`, and `scale`) using Ultralytics YOLOv8? That would help you empirically find the best values for your dataset.

Prompt:

how do i do the built in yolo tuning in python

ChatGPT said:

Perfect question 🙌 — YOLO's **built-in hyperparameter tuning** (added in **Ultralytics YOLOv8+**) can be run **directly in Python**, not just from the CLI.

Here's a full breakdown of how to do it in a **Python script or Colab notebook**, including code examples and best practices 📌

1. Install & Import YOLO

Make sure you're on the latest Ultralytics version:

- `!pip install -U ultralytics`

Then import:

- `from ultralytics import YOLO`

2. Load Your Model

You can start from a pretrained checkpoint or a custom `.pt` file:

- `model = YOLO("yolov8n.pt")` # or `yolov8s.pt` / `yolov8m.pt` / `your_custom_model.pt`

3. Run Built-In Hyperparameter Tuning

You can launch YOLO's automated tuner directly in Python using:

- `results = model.tune(`
- `data="data.yaml",` # path to your dataset YAML file
- `epochs=50,` # number of epochs for each trial
- `imgsz=640,` # input image size
- `iterations=20,` # number of tuning trials (default = 10)
- `batch=16,` # batch size
- `optimizer="AdamW",` # or "SGD"
- `)`

👉 This performs **Bayesian optimization** over key hyperparameters such as: `lr0`, `momentum`, `weight_decay`, `scale`, `translate`, `flip_lr`, `hsv_*`, etc.

Each trial runs a small training run, records mAP and loss, and saves the best-performing hyperparameter set.

4. Check and Use Results

After tuning, YOLO saves results in a new folder (usually `runs/tune/exp/`):

- `results`

This prints a summary, and you'll find a file:

- `runs/tune/exp/results.csv`
- `runs/tune/exp/best_hyperparameters.yaml`

You can load the best hyperparameters:

- `best_params = results.best_params`
- `print(best_params)`

Then use them in a new training run:

- `model.train(`
- `data="data.yaml",`
- `epochs=200,`

```
-     imgsz=640,  
-     batch=16,  
-     **best_params # apply tuned hyperparameters  
- )
```

🧩 5. Example Full Colab Block

```
- # Install Ultralytics  
- !pip install -U ultralytics  
-  
- from ultralytics import YOLO  
-  
- # Load model  
- model = YOLO("yolov8s.pt")  
-  
- # Run tuning  
- results = model.tune(  
-     data="data.yaml",  
-     epochs=50,  
-     imgsz=640,  
-     iterations=15,  
-     batch=16  
- )  
-  
- # Print best hyperparameters  
- print("\nBest found hyperparameters:")  
- print(results.best_params)  
-  
- # Train with tuned parameters  
- model.train(  
-     data="data.yaml",  
-     epochs=200,  
-     imgsz=640,  
-     batch=16,  
-     **results.best_params  
- )
```

⚡ Tips

- **iterations** controls how many different parameter sets are tested (each runs a short training).
 - The tuner uses **Optuna** internally — you can monitor logs for each trial in [runs/tune/](#).
 - To speed things up, lower **epochs** during tuning, then retrain longer afterward.
 - Works with **custom datasets**, as long as your [data.yaml](#) is correctly set up.
-

Would you like me to show you **how to set up the [data.yaml](#) file** (the structure and paths YOLO expects for tuning/training)?

- how to predict image detection on a test set
 - Excellent — once you've trained or fine-tuned a **YOLO model** (like YOLOv8 or YOLOv9), you can use it to **run object detection (prediction) on your test set** directly in Python or from the command line.
 - Below I'll walk you through both approaches (focusing on Python, since that's what you're using).
- ---
- 🧠 1. Load the Trained YOLO Model
 - First, make sure you have Ultralytics installed:
 - `!pip install -U ultralytics`

```
from ultralytics import YOLO
```

```
model = YOLO("runs/detect/train/weights/best.pt") # path to your trained model
```

 Option A: Folder of Test Images

```
results = model.predict(
    source="datasets/mydata/test/images", # folder or glob pattern
    save=True,                             # save predicted images
    save_txt=True,                         # save bounding boxes to .txt
    conf=0.25,                            # confidence threshold
    iou=0.45,                             # NMS IoU threshold
    imgsz=640,                            # inference image size
    device=0                              # or 'cpu'
)
```

runs/detect/predict/

```
result = model("datasets/mydata/test/images/example.jpg")
```

```
result.show()    # displays the annotated image in Colab
result.save()    # saves to runs/detect/predict/
```

```
for box in result[0].boxes:
    cls = int(box.cls)
    conf = float(box.conf)
    xxyy = box.xyxy.tolist()[0]
    print(f"Class: {model.names[cls]}, Conf: {conf:.2f}, BBox: {xxyy}")
```

If you already defined your test set in `data.yaml`, YOLO can automatically detect on that split:

This uses the paths defined under the `test:` key in your YAML file.

Each result object includes structured outputs:

```
results[0].boxes.xyxy # tensor of [x1, y1, x2, y2] per detection
results[0].boxes.conf # confidence scores
results[0].boxes.cls   # class indices
results[0].names       # class name mapping
```

If you want quantitative evaluation (mAP, precision, recall):

```
metrics = model.val(data="data.yaml", split="test")
print(metrics.box.map)           # mAP@0.5
print(metrics.box.map50)         # mAP@0.5
print(metrics.box.map75)         # mAP@0.75
```

This runs inference and computes standard detection metrics.

-
-  5. Command-Line Equivalent
 - If you prefer, you can also do it from the terminal (Colab shell):
 - `yolo predict model=runs/detect/train/weights/best.pt source=datasets/mydata/test/images save=True`
-

- ⚡ Bonus: Batch Processing + Results Table
- If you want to analyze detections across your test set programmatically:
- `import pandas as pd`

```
all_dets = []
for r in results:
    for box in r.bboxes:
        all_dets.append({
            "image": r.path,
            "class": model.names[int(box.cls)],
            "conf": float(box.conf),
            "x1": float(box.xyxy[0][0]),
            "y1": float(box.xyxy[0][1]),
            "x2": float(box.xyxy[0][2]),
            "y2": float(box.xyxy[0][3]),
        })

df = pd.DataFrame(all_dets)
df.head()
```

-
- Would you like me to include a **Colab-ready block** that automatically runs detection on your test set folder and saves all bounding box results into a **CSV file** (image name, class, confidence, coordinates)?
 - how do i access the iou score

Excellent question — the **IoU (Intersection over Union)** score can refer to two different contexts in YOLO depending on *what* you want:

1. **IoU during evaluation** (to measure performance) 
2. **IoU between specific boxes** (to compare detections manually) 

Let's go through both 

1. IoU During Evaluation (Validation or Test)

When you run:

```
metrics = model.val(data="data.yaml", split="test")
```

YOLO calculates and returns a full set of metrics including **IoU-based mAP values**.

You can access them like this:

```
print("mAP@0.5:", metrics.box.map50)
print("mAP@0.5:0.95:", metrics.box.map)
print("Precision:", metrics.box.mp)
print("Recall:", metrics.box.mr)
```

Explanation:

- `map50` → mAP at IoU threshold = 0.5
- `map` → averaged mAP across IoU thresholds from 0.5 to 0.95 (COCO metric)
- Internally, YOLO computes IoU between predicted and ground-truth boxes to determine matches.

If you want per-class IoUs:

- `metrics.box.maps` # list of per-class mAP values

2. IoU Between Specific Boxes (Manual Calculation)

If you want to manually compute IoU between two bounding boxes (for example, a prediction vs. ground truth), you can do it directly in Python.

Example:

```
- import torch
- from ultralytics.utils.metrics import bbox_iou
-
- # Example: predicted box and ground-truth box
- # Format: [x1, y1, x2, y2]
- pred_box = torch.tensor([100, 150, 200, 250])
- true_box = torch.tensor([120, 170, 220, 270])
-
- # Compute IoU
- iou = bbox_iou(pred_box.unsqueeze(0), true_box.unsqueeze(0), xywh=False)
- print("IoU:", iou.item())
```

Or, if you have a list of detections:

- `iou = bbox_iou(pred_boxes, true_boxes, xywh=False)`

3. IoU Threshold Used During Inference (NMS)

When you run predictions:

- `model.predict(source="test/images", iou=0.45)`

the `iou` parameter controls the **Non-Maximum Suppression (NMS)** threshold — how much overlap is allowed between predicted boxes for the same class before one is removed.

This is *not* a reported metric, but it influences final detections.

Summary

Use Case	How to Get IoU
Model performance	<code>metrics = model.val(...); metrics.box.map50</code>
IoU for each class	<code>metrics.box.maps</code>
Compare two boxes manually	<code>bbox_iou(pred_box, true_box)</code>

Control NMS
threshold

`model.predict(iou=0.45)`

Would you like me to show you how to **visualize IoU scores per class** (e.g., a bar chart of IoU per category from your test set)?

- should you tune before or after training

Excellent question — this one trips up a lot of people working with YOLO or any deep learning pipeline 🙌

🧠 Short Answer

You **tune before your final training**, but **after initial testing**.

That is:

1. ✅ **Train once with default hyperparameters** → get a baseline.
 2. ✅ **Tune** on that same dataset → find better hyperparameters.
 3. ✅ **Retrain from scratch** using the tuned parameters.
-

🔍 Why

Tuning is meant to **discover the best hyperparameters** (like learning rate, momentum, augmentations, weight decay, etc.) *for your dataset*.

It doesn't change the model weights permanently — each tuning trial runs a small “mini training” to test different combinations, then reports the best set.

So, tuning before your final training ensures your final model benefits from optimal hyperparameters.

🧩 Correct Workflow Example

Step 1. Initial Train (Baseline)

```
- model.train(  
-     data="data.yaml",  
-     epochs=50,  
-     imgsz=640,  
-     batch=16  
- )
```

👉 See what performance you get (mAP, precision, recall).

Step 2. Tune Hyperparameters

```
- results = model.tune(  
-     data="data.yaml",  
-     epochs=20,      # short runs for tuning  
-     imgsz=640,  
-     iterations=15  
- )
```

👉 This finds your best parameters.

Step 3. Final Train (With Tuned Params)

```
- model.train(  
-     data="data.yaml",  
-     epochs=200,  
-     imgsz=640,  
-     batch=16,  
-     **results.best_params  
- )
```

👉 This produces your *optimized* model.

⚠️ Don't

- Don't tune **after** training and expect it to "fix" an existing model — tuning doesn't alter already trained weights.
- Don't tune on the **test set** — always tune on training or validation splits to avoid overfitting to evaluation data.

🧠 Rule of Thumb

Train once → tune → retrain

This gives you both a baseline and a performance-optimized final model.

Would you like me to show you a **Colab-ready pipeline** that automatically runs this full sequence (baseline → tune → retrain → evaluate)?

-